

PROJECT_CONTEXT

Purpose

Build a lightweight personal “send pipe” to transfer files, text, and clipboard between a user’s own devices over peer-to-peer WebRTC DataChannels, using only a minimal signaling server for discovery and connection setup.

Architecture Overview

- `server/`: WebSocket signaling server (presence, pairing sessions, SDP/ICE relay)
- `app/`: Desktop app (Tauri shell + Vite/React UI), identity, pairing, WebRTC, transfers
- `shared/`: Shared protocol types + small helpers (safety phrase, chunk framing)

Current Phase

- App shell started: Tauri wrapper around the UI with native save-to-Downloads for received files.
- Account mode started: Google device-flow login on the signaling server; WS presence is scoped per user.
- LAN discovery started: server responds to UDP discovery; app auto-finds the local signaling server in Tauri.
- Dev convenience: Tauri can auto-start the Node signaling server if none is found on LAN (spawns `npm run dev -w @sendpipe/server`).
- Auth gating: when LAN server requires auth, the app delays opening the WebSocket until after sign-in.

Key Design Decisions

- WebSocket signaling is JSON messages with explicit `type` fields.
- Pairing uses short numeric codes and “safety phrase” verification derived from exchanged public keys.
- WebRTC uses STUN only; if connection fails, the UI reports it.
- DataChannel uses JSON control messages plus binary framed file chunks.
- File receive prefers native write to Downloads via a Tauri command; web download fallback remains for dev.
- Dev simulation supports multiple identities via URL `?profile=...` (namespaces browser storage keys).
- Optional account mode: server issues session tokens after Google device login; clients send `authToken` in `hello`; presence and signaling are scoped to the same `user.sub`.
- Local auth option for testing: email+password endpoints issue the same session token used by WS auth.
- Portable local auth option for LAN: clients derive `authToken = local:<sha256(email:password)>` so any server can scope the same account without shared DB/sessions.
- LAN discovery uses UDP broadcast to find a signaling server on the same network (Tauri only; web dev still uses env URLs).

File Structure

- `shared/src/protocol.ts`: WS + DataChannel message types
- `shared/src/safetyPhrase.ts`: safety phrase derivation
- `shared/src/chunkFraming.ts`: binary chunk header framing
- `server/src/index.ts`: signaling server
- `app/src/lib/\*`: client modules (identity, ws, webrtc, transfer)
- `app/src-tauri/src/lib.rs`: native commands (save to Downloads)
- `app/src-tauri/src/commands.rs`: native commands (save to Downloads)
- `app/src-tauri/icons/icon.png`: placeholder app icon
- `app/src/lib/authClient.ts`: Google device-flow helper (server endpoints)
- `app/src/lib/discoveryClient.ts`: Tauri LAN signaling discovery helper

Known Issues / TODO

- No TURN; strict NATs may fail.

- Trust model is minimal (no signatures yet); pairing safety phrase is informational only.
- Clipboard sync is stubbed; requires loop prevention and per-OS integration for “desktop”.
- Integration tests not added yet.
- App device-name edits must not restart signaling; name changes now only send a new `hello` message.
- Running “two devices” in the same browser window requires different `?profile=` values; otherwise they share a `deviceId` and will overwrite each other server-side.
- Tauri permissions/capabilities are not locked down yet (needs an explicit allowlist later).
- Tauri scripts run via `npx tauri ...` (no global `tauri` required).
- Auth uses Google `tokeninfo` (simple MVP verification). Replace with JWKS verification (or a proper auth service) before shipping.
- Local auth is for testing only (no reset/email verification/rate limiting).
- Portable local auth is “shared secret”; anyone with the same email+password can impersonate. Good for LAN testing, not a production auth system.
- Tauri “auto-start server” is dev-only (depends on Node/npm and repo layout). Shipping needs a hosted server or an embedded native signaling server.

Context Storage

`PROJECT\_CONTEXT.md` is the editable source of truth. `PROJECT\_CONTEXT.pdf` is regenerated from it so long context can be stored/read outside the chat window.

Planning Docs

App-first architecture plan rewrite lives at `docs/WEBRTC\_SEND\_PIPE\_ARCH\_PLAN\_v2.md` and is exported to `docs/WEBRTC\_SEND\_PIPE\_ARCH\_PLAN\_v2.pdf`.